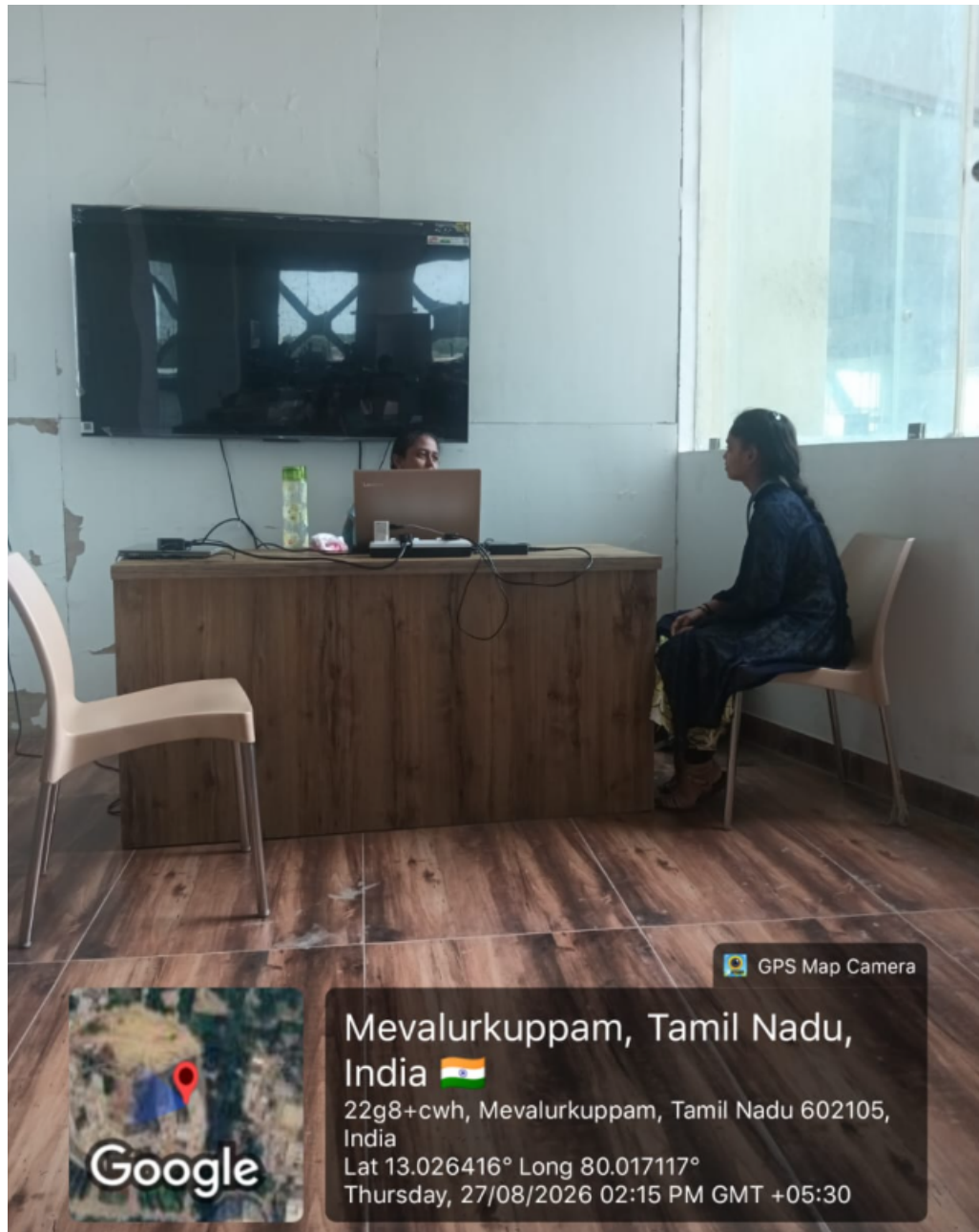


CO5 AT3 -Mock Interview



Question 1 — Query Processing in DBMS

Interview Question:

“Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?”

Answer:

Query processing is the process of converting a high-level SQL query into an efficient sequence of operations that can be executed by the DBMS to produce the required result.

The main steps are:

1. Query Parsing and Translation

The DBMS first **parses the SQL query** to check its syntax and semantics.

For example:

```
SELECT name FROM Student WHERE marks > 80;
```

The DBMS checks whether the query is correctly written and whether the table and column names exist. Then, it converts the query into an internal representation such as a **relational algebra expression**.

2. Query Optimization

The DBMS generates different possible ways to execute the query. These are called **execution plans**.

For example, it may decide whether to:

- Perform a full table scan
- Use an index
- Apply selection before a join
- Change the order of joins

The **query optimizer** compares these alternatives and chooses an efficient plan.

3. Query Execution

After selecting the best execution plan, the DBMS sends it to the **query execution engine**. The execution engine performs the required operations such as scanning, filtering, joining, and sorting.

How does the optimizer choose the best plan?

The optimizer estimates the **cost** of each possible execution plan using factors such as:

- **Disk I/O** – number of pages that need to be read/written
- **CPU cost** – processing required
- **Memory usage**

- **Number of records** expected as output
- **Availability of indexes**
- **Join methods and join order**

The plan with the **lowest estimated cost** is generally selected.

Simple Flow:

SQL Query → Parsing & Translation → Generate Execution Plans → Cost Estimation → Select Best Plan → Execute Query → Result

Interview Conclusion:

“So, query processing converts an SQL query into an executable plan. The query optimizer evaluates different execution plans based mainly on estimated I/O, CPU, memory, indexes, and intermediate-result sizes, and selects the plan with the lowest estimated cost.”

Question 2 — Heuristic vs Cost-Based Optimization

Interview Question:

“What is the difference between heuristic optimization and cost-based optimization? When would you use each?”

Answer:

Query optimization is the process of selecting an efficient execution plan for a SQL query.

There are two important approaches:

1. Heuristic Optimization

Heuristic optimization uses a set of predefined **rules or guidelines** to improve a query without calculating the exact cost of every possible execution plan.

Common rules include:

- Perform **selection early** to reduce the number of records.
- Perform **projection early** to reduce the number of columns.
- Perform smaller operations before expensive operations.
- Reduce the size of intermediate results.

Example:

If a query contains a condition such as marks > 80, the DBMS can filter the records first and then perform a join. This reduces the number of records involved in the join.

Advantage: Fast and simple.

Limitation: It may not always produce the globally best execution plan.

2. Cost-Based Optimization

Cost-based optimization generates multiple possible execution plans and estimates the **cost** of each plan.

The cost can depend on:

- Disk I/O
- CPU processing
- Memory usage
- Number of records
- Available indexes
- Join methods and join order

The optimizer selects the plan having the **lowest estimated cost**.

Example:

For a large table, the optimizer may compare a full table scan with an index scan and select whichever has the lower estimated cost.

Difference

Heuristic Optimization	Cost-Based Optimization
Uses predefined rules	Uses estimated execution costs
Faster optimization	Usually requires more optimization time
Does not extensively compare plans	Compares multiple execution plans
Simple to implement	More complex
May not give the optimal plan	Usually gives a better plan based on cost estimates

When to use each?

- **Heuristic optimization:** Useful when quick and simple optimization is required, especially for straightforward queries.
- **Cost-based optimization:** Preferred for **complex queries**, especially queries involving multiple joins, indexes, and large datasets, where choosing the right execution plan significantly affects performance.

Interview Conclusion:

“Heuristic optimization applies predefined rules to simplify and improve a query quickly, whereas cost-based optimization evaluates different execution plans and chooses the one with the lowest estimated cost. Heuristics are useful for quick optimization, while cost-based optimization is preferred for complex and large database queries.”

Question 3 — Database Connectivity: JDBC vs ODBC

Interview Question:

“How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.”

Answer:

Database connectivity means establishing a connection between an application and a database so that the application can **insert, retrieve, update, and delete data**.

Steps to implement database connectivity

In a real project, the following steps are generally followed:

1. **Load/Register the database driver**
The appropriate driver is made available to the application.
2. **Establish a connection**
The application connects to the database using the database URL, username, and password.
3. **Create a statement**
A statement or prepared statement is created to execute SQL commands.
4. **Execute SQL query**
SQL operations such as SELECT, INSERT, UPDATE, and DELETE are executed.
5. **Process the result**
For a SELECT query, the returned records are processed by the application.
6. **Close resources**
The result set, statement, and database connection are closed after completing the operation.

JDBC

JDBC (Java Database Connectivity) is a Java-based API that allows Java applications to communicate with databases.

For example, a Java application can use JDBC to connect to **MySQL, Oracle, PostgreSQL**, etc., using the appropriate JDBC driver.

ODBC

ODBC (Open Database Connectivity) is a standard database connectivity interface that allows applications to communicate with different database systems through ODBC drivers.

JDBC vs ODBC

JDBC

Java-specific API

Mainly used by Java applications

ODBC

Language-independent standard

Can be used by applications written in different

	languages
Uses JDBC drivers	Uses ODBC drivers
Designed specifically for Java	General database connectivity interface
Commonly used in Java-based projects	Useful for applications requiring a common database interface

Real-World Example

Suppose we develop a **college student management system** in Java. The application needs to retrieve student details from a MySQL database.

The application would:

Java Application → JDBC Driver → MySQL Database → Student Data

For example, when a user searches for a student ID, the Java application sends an SQL query through JDBC, receives the student's details, and displays them.

Interview Conclusion

“In a real project, database connectivity involves loading the driver, establishing a connection, creating a statement, executing SQL queries, processing results, and closing resources. JDBC is specifically designed for Java applications, whereas ODBC provides a general database connectivity interface for applications using different programming languages.”

Question 4 — ACID Properties

Interview Question:

“What are ACID properties? Give a real-world example where each property is critical.”

Answer:

ACID properties are four important properties of a database transaction that ensure **data consistency, reliability, and correctness**.

ACID stands for:

A – Atomicity

C – Consistency

I – Isolation

D – Durability

1. Atomicity

Atomicity means a transaction is treated as a **single unit**. Either all operations of the transaction are completed successfully, or none of them are performed.

Example:

Consider transferring ₹5,000 from Account A to Account B.

- ₹5,000 is deducted from A.
- ₹5,000 is added to B.

If money is deducted from A but cannot be added to B because of an error, the entire transaction is **rolled back**.

So, “**all or nothing**” represents Atomicity.

2. Consistency

Consistency ensures that a transaction changes the database from **one valid state to another valid state** while maintaining all database rules and constraints.

Example:

Suppose a bank account cannot have a negative balance. If a transaction tries to withdraw more money than the available balance, the database should reject the transaction.

3. Isolation

Isolation means that multiple transactions executing at the same time should not incorrectly interfere with one another.

Example:

Suppose two customers try to purchase the **last available seat** on a train at the same time.

Isolation ensures that both transactions do not successfully reserve the same seat.

4. Durability

Durability means that once a transaction is successfully committed, its changes are **permanently stored**, even if a system failure occurs afterward.

Example:

Suppose you successfully transfer ₹5,000 through online banking and receive a transaction confirmation. Even if the database server crashes immediately afterward, the completed transaction should not be lost.

Quick Summary

Property	Meaning	Real-World Example
Atomicity	All operations happen or none happen	Bank money transfer
Consistency	Database remains valid	Maintaining account constraints
Isolation	Concurrent transactions don't interfere incorrectly	Booking the last train seat
Durability	Committed data is permanently saved	Successful online payment

Interview Conclusion

“ACID properties ensure reliable transaction processing in DBMS. Atomicity provides all-or-nothing execution, Consistency maintains valid database states, Isolation prevents interference between concurrent transactions, and Durability ensures committed changes are permanently stored.”

Question 5 — Concurrency Control Problems and 2PL

Interview Question:

“Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?”

Answer:

Concurrency control is a technique used in DBMS to manage multiple transactions executing at the same time while maintaining **data consistency and correctness**.

Three common concurrency problems are:

1. Dirty Read

A **dirty read** occurs when one transaction reads data that has been **modified by another transaction but not yet committed**.

Example:

- Transaction T1 changes an account balance from ₹10,000 to ₹8,000.
- Before T1 commits, T2 reads the balance as ₹8,000.
- T1 then fails and rolls back to ₹10,000.
- T2 has already read an incorrect value.

So, T2 has read **uncommitted data**.

2. Lost Update

A **lost update** occurs when two transactions read and update the same data, causing one transaction's update to **overwrite the other transaction's update**.

Example:

Suppose the balance is ₹10,000.

- T1 reads ₹10,000.
- T2 also reads ₹10,000.
- T1 updates it to ₹12,000.
- T2 updates it to ₹11,000.

The update made by T1 is lost because T2 overwrites it.

3. Unrepeatable Read

An **unrepeatable read** occurs when a transaction reads the same record twice but gets **different values** because another transaction modified the record between the two reads.

Example:

- T1 reads salary = ₹30,000.
- T2 changes salary to ₹35,000 and commits.
- T1 reads the same salary again and gets ₹35,000.

Thus, T1 gets different values for the same record during one transaction.

How Does 2PL Solve These Problems?

2PL (Two-Phase Locking) is a concurrency control protocol that uses locks to ensure **conflict-serializable schedules**.

It has two phases:

Phase 1 — Growing Phase

The transaction can **acquire locks**, but it cannot release any lock.

Phase 2 — Shrinking Phase

The transaction can **release locks**, but it cannot acquire any new locks.

There are two important types of locks:

- **Shared Lock (S-lock):** Used for reading data.
- **Exclusive Lock (X-lock):** Used for modifying data.

Example

If T1 wants to update a bank balance, it obtains an **exclusive lock** on that record.

While T1 holds the exclusive lock:

- T2 cannot modify the same record.
- Depending on the locking protocol and lock type, T2 may have to wait before reading or modifying it.

This prevents conflicting operations from occurring incorrectly.

Summary

Problem	Meaning	How Locking Helps
Dirty Read	Reading uncommitted data	Locks prevent conflicting access
Lost Update	One update overwrites another	Exclusive locks protect updates
Unrepeatable Read	Same read gives different values	Appropriate locks maintain controlled access

Interview Conclusion

“Concurrency control prevents multiple transactions from producing inconsistent results. Dirty read means reading uncommitted data, lost update means one update overwrites another, and unrepeatable read means repeated reads return different values. Two-Phase Locking controls these conflicts using shared and exclusive locks and ensures conflict serializability.”

Question 6 — Deadlock in DBMS

Interview Question:

“What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.”

Answer:

A **deadlock** occurs in a DBMS when two or more transactions are **waiting for each other to release resources**, and none of them can proceed.

Example

Suppose there are two transactions, **T1** and **T2**:

- T1 holds a lock on resource **A** and waits for **B**.
- T2 holds a lock on resource **B** and waits for **A**.

So:

T1 → waits for B → T2

T2 → waits for A → T1

Neither transaction can continue. This situation is called a **deadlock**.

Deadlock Detection Algorithm

The DBMS can detect deadlocks using a **Wait-For Graph**.

Steps:

1. Create a node for each transaction.
2. Draw an edge **T_i → T_j** if transaction **T_i** is waiting for a resource held by **T_j**.
3. Check the graph for a **cycle**.
4. If a cycle exists, a deadlock is present.
5. The DBMS selects one transaction as a **victim**, rolls it back, and releases its locks.
6. The other transaction can then continue.

Example:

If the graph contains:

T1 → T2 → T1

there is a cycle, so **T1 and T2 are in deadlock**.

Deadlock Prevention Strategies

1. Resource Ordering

Assign a fixed order to resources and require transactions to request resources only according to that order.

For example:

$A \rightarrow B \rightarrow C$

A transaction must request A before B and B before C.

This prevents **circular waiting**, which is necessary for deadlock.

2. Timeout

A transaction is allowed to wait for a limited amount of time.

If it waits longer than the specified time, the DBMS assumes a deadlock may have occurred and **aborts or rolls back the transaction**.

Other Strategy

Wait-Die / Wound-Wait:

These timestamp-based techniques decide whether a transaction should wait or be rolled back based on the transaction's age.

Quick Summary

Concept	Explanation
Deadlock	Transactions wait for each other indefinitely
Detection	Use a Wait-For Graph
Cycle	Indicates deadlock
Victim	A transaction selected for rollback
Resource Ordering	Prevents circular waiting
Timeout	Aborts a transaction after excessive waiting

Interview Conclusion

“Deadlock occurs when transactions wait indefinitely for resources held by each other. DBMS can detect it using a Wait-For Graph by checking for cycles. Deadlocks can be prevented using resource ordering, timeout techniques, and timestamp-based methods such as wait-die and wound-wait.”

Question 7 — Immediate Update vs Deferred Update

Interview Question:

“Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?”

Answer:

Recovery in DBMS is the process of restoring the database to a **consistent state after a transaction failure or system crash**.

Two important recovery techniques are **Immediate Update** and **Deferred Update**.

1. Immediate Update

In **Immediate Update**, changes made by a transaction can be written to the database **before the transaction commits**.

Because uncommitted changes may already be stored in the database, the DBMS maintains a **log** to recover the database if the transaction fails.

If a transaction fails before committing, its changes must be **undone (UNDO)**.

Example:

Suppose T1 changes:

Balance = ₹10,000 → ₹8,000

The value ₹8,000 may be written to the database before T1 commits. If T1 fails, the DBMS uses the log to restore the balance to ₹10,000.

Main feature: Requires **UNDO**, and depending on the recovery scheme, REDO may also be required.

2. Deferred Update

In **Deferred Update**, changes made by a transaction are **not written to the database until the transaction commits**.

The changes are initially maintained in the log or temporary storage.

If the transaction fails before committing, its changes do not need to be undone because they were never written to the database.

After a successful commit, the changes are applied to the database.

Example:

T1 changes the balance from ₹10,000 to ₹8,000.

The database continues to contain ₹10,000 until T1 commits. After a successful commit, ₹8,000 is written to the database.

Main feature: Generally requires **REDO**, but not UNDO for uncommitted transactions.

Difference Between Immediate and Deferred Update

Immediate Update

Changes may be written before commit

Uncommitted changes may exist in database

Requires UNDO for failed transactions

Recovery is more complex

Suitable when updates need to be written during transaction execution

Deferred Update

Changes are written after commit

Uncommitted changes are not written to database

No UNDO for uncommitted transactions

Recovery is comparatively simpler

Suitable when simpler recovery is preferred

When is Each Preferred?

Immediate Update:

Preferred when the system needs to write database changes during transaction execution and can manage the additional recovery requirements.

Deferred Update:

Preferred when a simpler recovery mechanism is desired and changes can be safely delayed until the transaction commits.

Interview Conclusion

“In immediate update, database changes can be written before commit, so recovery may need to undo uncommitted changes. In deferred update, changes are applied only after commit, making recovery simpler because uncommitted changes do not reach the database. The choice depends on the system's performance and recovery requirements.”

Question 8 — Shadow Paging

Interview Question:

“Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?”

Answer:

Shadow Paging is a database recovery technique used to maintain database consistency after a **transaction failure or system crash**.

It works by maintaining **two page tables**:

- **Shadow page table**
- **Current page table**

How Shadow Paging Works

1. When a transaction starts, the DBMS creates a **shadow page table** that points to the original database pages.

2. The transaction does not modify the original pages directly.
3. When a page needs to be modified, a **new copy of that page** is created.
4. The current page table is updated to point to the modified page.
5. The shadow page table continues to point to the original pages.
6. If the transaction **commits**, the current page table becomes the new valid page table.
7. If the transaction **fails**, the DBMS can discard the modified pages and use the shadow page table to restore the previous state.

Simple Representation

Before Transaction:

Shadow Page Table → Original Pages

During Transaction:

Current Page Table → Modified/New Pages

Shadow Page Table → Original Pages

Commit:

Current page table becomes permanent.

Failure:

Discard modified pages and use the shadow page table.

Advantages of Shadow Paging

1. **No complex log management** is required.
2. **Fast recovery** because the DBMS can return to the shadow page table.
3. No need to perform traditional **UNDO/REDO operations**.
4. It provides a simple method for restoring the previous database state after failure.

Limitations of Shadow Paging

1. **Extra storage** may be required because modified pages are copied.
2. Maintaining page tables can create **overhead**.
3. It can cause **fragmentation** because new pages may be allocated at different locations.
4. It may not be efficient for databases with **frequent updates**.
5. Log-based recovery is generally more flexible for large and complex database systems.

Shadow Paging vs Log-Based Recovery

Shadow Paging

Log-Based Recovery

Uses shadow and current page tables	Uses a transaction log
Recovery uses the shadow page table	Recovery uses UNDO/REDO operations
No traditional log required	Requires log records
Recovery can be fast	Recovery depends on log processing
Page copying can cause overhead	Log management creates overhead
Can suffer from fragmentation	Generally more flexible for large systems

Interview Conclusion

“Shadow paging maintains a shadow copy of the page table and creates new copies of modified pages. If a transaction commits, the new page table is accepted; if it fails, the shadow page table restores the previous state. Its main advantage is simple and fast recovery, while its limitations include extra storage, page-copying overhead, and fragmentation compared with log-based recovery.”

Question 9 — Serializability and Precedence Graph

Interview Question:

“What is a serializability schedule? How do you test for conflict serializability using a precedence graph?”

Answer:

A **serial schedule** is a schedule in which transactions are executed **one after another**, without overlapping.

A **serializable schedule** is a concurrent schedule whose final result is **equivalent to some serial schedule**. Serializability is important because it ensures that concurrent transactions maintain database consistency.

Conflict Serializability

A schedule is **conflict serializable** if it can be transformed into a serial schedule by swapping **non-conflicting operations**.

Two operations are conflicting when:

- They belong to **different transactions**.
- They access the **same data item**.
- At least one operation is a **write**.

Examples of conflicts:

- Read–Write (R-W)
- Write–Read (W-R)
- Write–Write (W-W)

Precedence Graph

A **precedence graph** (also called a serialization graph) is used to test whether a schedule is conflict serializable.

Steps:

1. Create a node for each transaction.

For transactions T1 and T2:

T1 T2

2. Identify conflicting operations.

If an operation of T1 occurs before a conflicting operation of T2 on the same data item, draw an edge:

T1 → T2

This means T1 must come before T2 in the equivalent serial schedule.

3. Check the graph for a cycle.

- No cycle → **Conflict serializable**
- Cycle exists → **Not conflict serializable**

Example

Consider the schedule:

T1 T2

Read(A)

Read(A)

Write(A)

Write(A)

Here, T1's Write(A) conflicts with T2's Read(A) and Write(A).

Therefore, we get:

T1 → T2

There is no cycle, so the schedule is **conflict serializable** and is equivalent to the serial order:

T1 → T2

Example of a Cycle

If the precedence graph contains:

T1 → T2 → T1

there is a **cycle**.

Therefore, the schedule is **not conflict serializable**.

Quick Summary

Condition	Result
No cycle in precedence graph	Conflict serializable
Cycle in precedence graph	Not conflict serializable
Edge $T_i \rightarrow T_j$	T_i must execute before T_j

Interview Conclusion

“A serializable schedule produces the same result as some serial execution of transactions. To test conflict serializability, we construct a precedence graph using transactions as nodes and conflicting operations as directed edges. If the graph has no cycle, the schedule is conflict serializable; if it contains a cycle, it is not conflict serializable.”

Question 10 — Two-Phase Locking (2PL) Variations

Interview Question:

“Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?”

Answer:

Two-Phase Locking (2PL) is a concurrency-control protocol used in DBMS to ensure **conflict serializability**. It divides a transaction into two phases:

1. **Growing Phase** – The transaction can acquire locks but cannot release them.
2. **Shrinking Phase** – The transaction can release locks but cannot acquire new locks.

There are three important variations of 2PL.

1. Basic 2PL

In **Basic 2PL**:

- A transaction can acquire locks during the growing phase.
- Once it releases its first lock, it enters the shrinking phase.
- It cannot acquire any new lock after that.

Advantage: Ensures conflict serializability.

Limitation: It can still result in **deadlocks**, and early release of locks can cause cascading rollbacks.

2. Strict 2PL

In **Strict 2PL**, all **exclusive (X) locks** are held until the transaction **commits or aborts**.

This prevents other transactions from reading or modifying data written by an uncommitted transaction.

Advantages:

- Prevents dirty reads of written data.
- Avoids cascading rollbacks.
- Makes recovery easier.

Strict 2PL is widely used in practical database systems because it provides good **concurrency control and recoverability**.

3. Rigorous 2PL

In **Rigorous 2PL**, both **shared (S) locks and exclusive (X) locks** are held until the transaction commits or aborts.

Therefore, no lock is released before the transaction finishes.

Advantage: Provides very strong isolation and makes the serialization order closely related to transaction commit order.

Limitation: Holding all locks for the entire transaction can reduce concurrency.

Comparison

Basic 2PL	Strict 2PL	Rigorous 2PL
Locks acquired in growing phase and released in shrinking phase	X-locks held until commit/abort	S-locks and X-locks held until commit/abort
Ensures conflict serializability	Ensures conflict serializability and easier recovery	Provides stronger locking discipline
May cause cascading rollback	Prevents cascading rollback	Prevents cascading rollback
More concurrency than stricter forms	Good balance of safety and concurrency	Can reduce concurrency
Can have deadlocks	Can have deadlocks	Can have deadlocks

Which is Most Commonly Used?

Strict 2PL is generally preferred in practical DBMS because it provides a good balance between **concurrency, serializability, and recoverability**. By holding exclusive locks until

commit or abort, it prevents other transactions from using uncommitted updates and simplifies recovery.

Interview Conclusion

“Basic 2PL separates lock acquisition and release into growing and shrinking phases. Strict 2PL keeps exclusive locks until commit or abort, while Rigorous 2PL keeps both shared and exclusive locks until the transaction finishes. Strict 2PL is commonly preferred because it provides strong consistency and recoverability while allowing better concurrency than Rigorous 2PL.”